



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRACTICA N°3

MODELADO GEOMÉTRICO

NOMBRE COMPLETO: García Cruz Andrea

N.º de Cuenta: 316172697

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 01

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 17/09/26

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Edición de Código

```
473 //NUEVA FUNCION COHETE
474 void DibujarCohete(glm::mat4 model, GLuint uniformModel, GLuint uniformColor)
475 {
476     // --- CUERPO (cilindro alargado) ---
477     glm::mat4 cuerpo = model;
478     cuerpo = glm::scale(cuerpo, glm::vec3(0.5f, 2.5f, 0.5f));
479     DibujarInstancia(meshList[2], cuerpo, glm::vec3(0.8f, 0.8f, 0.8f), uniformModel, uniformColor); // gris
480
481     // --- PUNTA (cono) ---
482     glm::mat4 punta = model;
483     punta = glm::translate(punta, glm::vec3(0.0f, 1.45f, 0.0f)); // antes 1.65f
484     punta = glm::scale(punta, glm::vec3(0.25f, 0.4f, 0.25f));
485     DibujarInstancia(meshList[3], punta, glm::vec3(1.0f, 0.0f, 0.0f), uniformModel, uniformColor);
486
487     // --- VENTANILLA (esfera) ---
488     glm::mat4 ventanilla = model;
489     ventanilla = glm::translate(ventanilla, glm::vec3(0.0f, 0.4f, 0.55f)); // antes 0.26f
490     ventanilla = glm::scale(ventanilla, glm::vec3(0.15f, 0.15f, 0.05f));
491     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(ventanilla));
492     glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.3f, 0.6f, 1.0f)));
493     sp.render();
494
495     // --- ALETAS (4 cubos escalados, alrededor de la base) ---
496     for (int i = 0; i < 4; i++)
497     {
498         glm::mat4 aleta = model;
499         aleta = glm::rotate(aleta, glm::radians(90.0f * i), glm::vec3(0.0f, 1.0f, 0.0f));
500         aleta = glm::translate(aleta, glm::vec3(0.6f, -1.0f, 0.0f));
501         aleta = glm::scale(aleta, glm::vec3(0.3f, 0.6f, 0.05f));
502         DibujarInstancia(meshList[0], aleta, glm::vec3(1.0f, 0.0f, 0.0f), uniformModel, uniformColor); // rojo
503     }
504 }
505
506 ///*****NUEVA FUNCION DE CILINDRO PARA COHETE
507 void CrearCilindro(int res, float R) {
508     vector<GLfloat> vertices;
509     vector<unsigned int> indices;
510
511     // Pared lateral: un punto abajo y uno arriba por segmento
512     for (int n = 0; n <= res; n++) {
513         float ang = n * (2 * PI / res);
514         float x = R * cos(ang);
515         float z = R * sin(ang);
516         vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
517         vertices.push_back(x); vertices.push_back(0.5f); vertices.push_back(z);
518     }
519     for (int n = 0; n < res; n++) {
520         unsigned int b0 = n * 2, t0 = n * 2 + 1;
521         unsigned int b1 = (n + 1) * 2, t1 = (n + 1) * 2 + 1;
522         indices.push_back(b0); indices.push_back(t0); indices.push_back(b1);
523         indices.push_back(t0); indices.push_back(t1); indices.push_back(b1);
524     }
525
526     // Tapa inferior
527     unsigned int centroInf = vertices.size() / 3;
528     vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
529     unsigned int inicioInf = vertices.size() / 3;
530     for (int n = 0; n <= res; n++) {
531         float ang = n * (2 * PI / res);
532         vertices.push_back(R * cos(ang)); vertices.push_back(-0.5f); vertices.push_back(R * sin(ang));
533     }
534     for (int n = 0; n < res; n++) {
535         indices.push_back(centroInf);
536     }
537 }
```

POSICIONA Y COLOREA LAS PARTES DEL COHETE POR BLOQUES

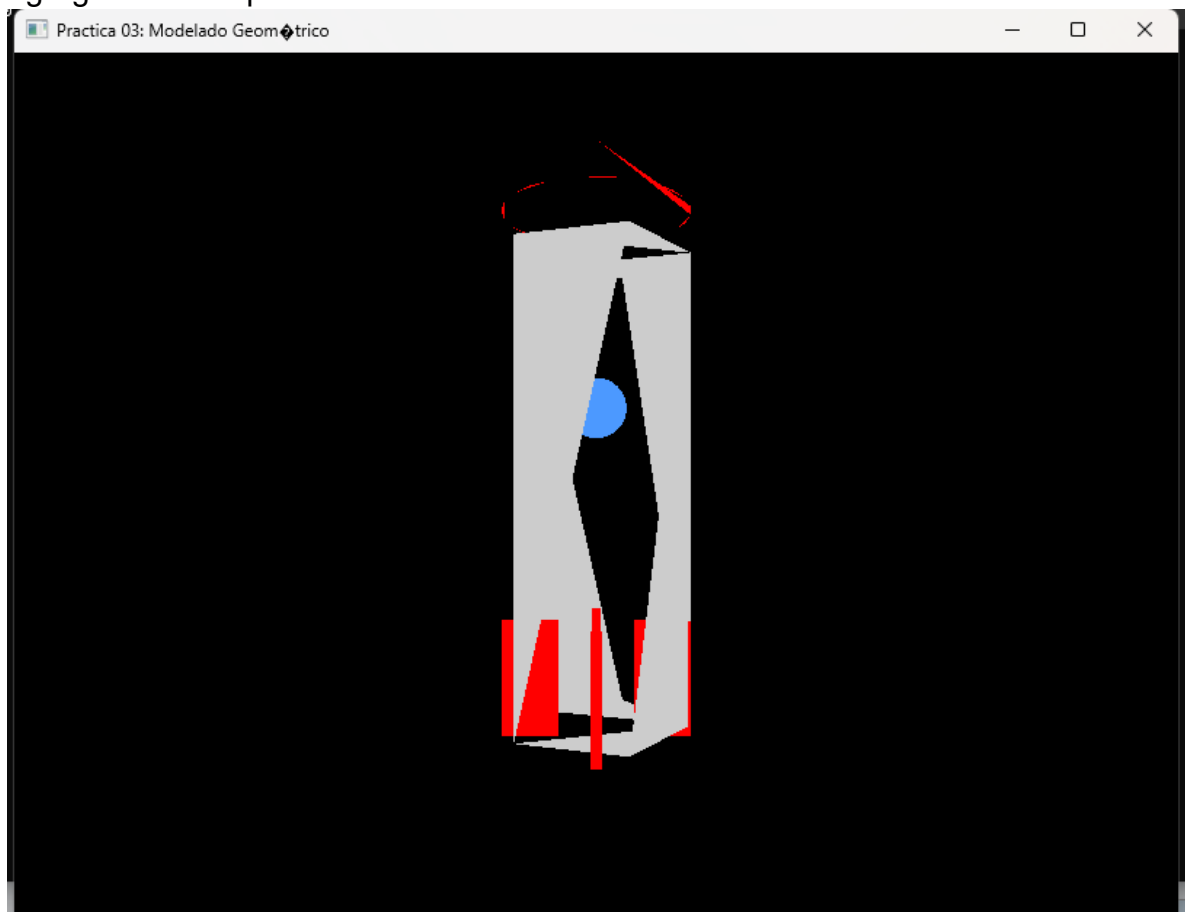
SE REESCRIBIO INDICES Y VERTICES ADEMAS DE AGREGAR TAPAS PARA QUE NO SE VIERA HUECO

```
Practica03 (Ámbito global) DibujarCohete(g)
411 // *****NUEVA FUNCION CONO DEL COHETE
412 void CrearCono(int res, float R) {
413     vector<GLfloat> vertices;
414     vector<unsigned int> indices;
415
416     unsigned int apice = 0;
417     vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);
418
419     unsigned int inicioBase = vertices.size() / 3;
420     for (int n = 0; n <= res; n++) {
421         float ang = n * (2 * PI / res);
422         vertices.push_back(R * cos(ang)); vertices.push_back(-0.5f); vertices.push_back(R * sin(ang));
423     }
424     for (int n = 0; n < res; n++) {
425         indices.push_back(apice);
426         indices.push_back(inicioBase + n);
427         indices.push_back(inicioBase + n + 1);
428     }
429
430     // Tapa de la base (para que no quede hueca)
431     unsigned int centroBase = vertices.size() / 3;
432     vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
433     unsigned int inicioBase2 = vertices.size() / 3;
434     for (int n = 0; n <= res; n++) {
435         float ang = n * (2 * PI / res);
436         vertices.push_back(R * cos(ang)); vertices.push_back(-0.5f); vertices.push_back(R * sin(ang));
437     }
438     for (int n = 0; n < res; n++) {
439         indices.push_back(centroBase);
440         indices.push_back(inicioBase2 + n + 1);
441
442     unsigned int apice = 0;
443     vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);
444
445     unsigned int inicioBase = vertices.size() / 3;
446     for (int n = 0; n <= res; n++) {
447         float ang = n * (2 * PI / res);
448         vertices.push_back(R * cos(ang)); vertices.push_back(-0.5f); vertices.push_back(R * sin(ang));
449     }
450     for (int n = 0; n < res; n++) {
451         indices.push_back(apice);
452         indices.push_back(inicioBase + n);
453         indices.push_back(inicioBase + n + 1);
454     }
455
456     // Tapa de la base (para que no quede hueca)
457     unsigned int centroBase = vertices.size() / 3;
458     vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
459     unsigned int inicioBase2 = vertices.size() / 3;
460     for (int n = 0; n <= res; n++) {
461         float ang = n * (2 * PI / res);
462         vertices.push_back(R * cos(ang)); vertices.push_back(-0.5f); vertices.push_back(R * sin(ang));
463     }
464     for (int n = 0; n < res; n++) {
465         indices.push_back(centroBase);
466         indices.push_back(inicioBase2 + n + 1);
467     }
468 }
```

Problemas presentados

- ❖ Cambios en la interfaz de Visual Studio tras una actualización, que dificultaron ubicar los botones de compilación y ejecución.
- ❖ El archivo de código se abrió como "Archivos varios" en lugar de cargarse como parte del proyecto/solución (.slnx), por lo que no aparecerían las opciones de compilación.
- ❖ Error de compilación por declaración múltiple de una variable shader (C2374/C2086), causado por una duplicación accidental al editar el archivo.
- ❖ Pantalla en negro al probar una malla de color por vértice: se debía a que el shader de color activo no coincidía con el fragment shader correspondiente, y a que ubicación del color uniform no se estaba actualizando.

- ❖ Figura completa ocupando toda la pantalla: la matriz de modelo no se trasladaba lejos del origen, dejando la cámara dentro de la geometría.
- ❖ Piezas del cohete que no rotaban con los controles de teclado por faltar ejes de rotación en la cadena de transformaciones.
- ❖ Error de compilación por número insuficiente de argumentos, causado por una llamada a función ubicada fuera del bucle de render, la sección de inicialización.
- ❖ Huecos visibles en el cilindro y el cono: los índices de estas primitivas estaban pensados para un modo de dibujo distinto al utilizado, lo que generaba triángulos mal formados; se corrigió regenerando los índices para el modo de dibujo correcto y agregando las tapas faltantes.



- ❖ Piezas del cohete mal ensambladas (ventanilla oculta dentro del cuerpo, hueco entre el cuerpo y la punta, aletas hundidas en el cilindro): se resolvió recalculando las posiciones en función de las dimensiones reales de cada primitiva tras aplicar su escala.

Conclusión

Se logró construir un cohete espacial mediante la instancia de primitivas geométricas ya existentes en el proyecto (cilindro, cono, cubo y esfera), reutilizando la estructura de clases y el patrón de dibujo de prácticas anteriores. El ejercicio permitió reforzar el manejo de matrices de transformación (traslación, rotación y escala) para el ensamblaje correcto de piezas compuestas, así como identificar y corregir errores de generación de geometría en primitivas creadas previamente. Los problemas encontrados fueron principalmente de dos tipos: errores de configuración del entorno de desarrollo y errores de lógica en el cálculo de posiciones y en la generación de índices de las mallas, ambos resueltos revisando el flujo de datos entre las funciones de creación y dibujo.

Bibliografía en formato APA

Anthropic. (2026). Claude (Sonnet 5)[Modelo de lenguajes de gran escala].
<https://claude.ai/>